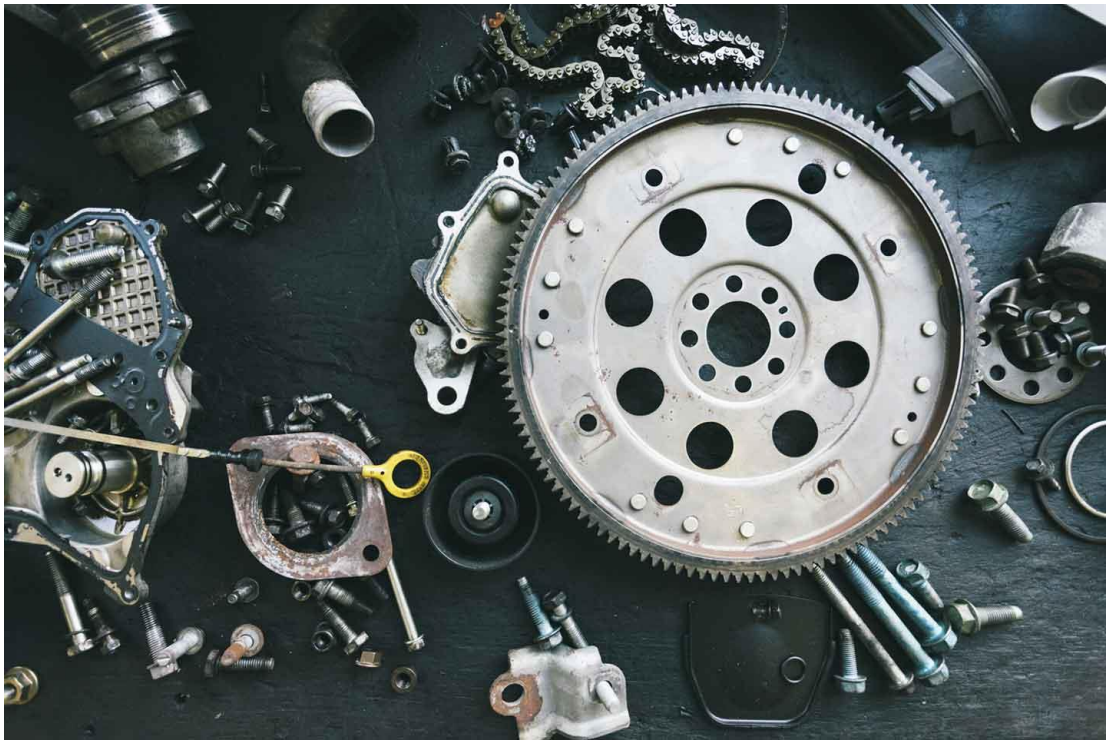
IONIC

Multilanguage



Progresa
Una formación de futuro

ÍNDICE

Primeros pasos 3

Archivos JSON 5

Servicio 6

¿Qué hemos hecho? 6

Tabs 7

Tab1 Tab2 Tab3 9

PRIMEROS PASOS

Para empezar vamos a crear el proyecto en el que vamos a trabajar con el multiidioma.

En la **terminal**:

```
Ionic start multilanguageApp blank

npm install @ngx-translate/core

npm install @ngx-translate/http-loader
```

Creamos el Proyecto de Ionic con **standalone**, esto quiere decir que trabajaremos sin módulos.

Para este proyecto vamos a necesitar:

- 4 páginas
- 1 servicio
- 1 componente
- 3 ficheros json

Vamos a empezar a crearlos en la **terminal**:

```
ionic g page pages/tabs -spec=false

ionic g page pages/tab1 -spec=false

ionic g page pages/tab2 -spec=false

ionic g page pages/tab3 -spec=false

ionic g s services/translate-config -skip-tests

ionic g c components/explore-container -spec=false
```

Los ficheros **json** los creamos a mano en nuestra carpeta **assets**, en una subcarpeta llamada **i18n**. Los llamaremos ar.json, en.json, es.json.

Para terminar vamos a añadir las configuraciones iniciales del ngx-translate. En nuestro fichero **main.ts:**

```
import { enableProdMode, importProvidersFrom } from '@angular/core';
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter } from '@angular/router';
import { IonicModule, IonicRouteStrategy } from '@ionic/angular';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';
import { environment } from './environments/environment';
import {TranslateLoader, TranslateModule} from "@ngx-
translate/core";
import {TranslateHttpLoader} from "@ngx-translate/http-loader";
import {HttpClient, HttpClientModule } from "@angular/common/http";

if (environment.production) {
  enableProdMode();
}

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    importProvidersFrom(IonicModule.forRoot({})),
    provideRouter(routes),
    importProvidersFrom(
      HttpClientModule,
      TranslateModule.forRoot({
        loader: {
          provide: TranslateLoader,
          useFactory: (createTranslateLoader),
          deps: [HttpClient]
        }
      })
    )
  ],
});
export function createTranslateLoader(http: HttpClient) {
  return new TranslateHttpLoader(http, './assets/i18n/', '.json');
}
```

ARCHIVOS JSON

Vamos a configurar nuestros archivos json.

En **ar.json**:

```
{
  "HOME": "منزل",
  "CONTENT": "محتوى",
  "ABOUT": "حول",
  "HELLO": "مرحبًا",
  "WELCOME": "مرحبًا بك في تطبيق Ionic متعدد اللغات",
  "SIMPLE_APP": "هذا تطبيق بسيط جدا",
  "MY_NAME": "اسمي فرانسيسكو بيلدا"
}
```

En **en.json**:

```
{
  "HOME": "Home",
  "CONTENT": "Content",
  "ABOUT": "About",
  "HELLO": "Hello",
  "WELCOME": "Welcome to the multilanguage Ionic app",
  "SIMPLE_APP": "This is just a simple app",
  "MY_NAME": "My name is Francisco Belda"
}
```

En **es.json**:

```
{
  "HOME": "Hogar",
  "CONTENT": "Contenido",
  "ABOUT": "Sobre",
  "HELLO": "Hola",
  "WELCOME": "Bienvenido a la aplicación multilenguaje de
Ionic",
  "SIMPLE_APP": "Esta es solo una aplicación simple.",
  "MY_NAME": "Mi nombre es Francisco Belda"
}
```

SERVICIO

Necesitamos un servicio para establecer el idioma por defecto, ver el idioma actual y establecer un idioma dentro de la App. En **translate-config.service.ts**:

```
import { Injectable } from '@angular/core';
import {TranslateService} from "@ngx-translate/core";

@Injectable({
  providedIn: 'root'
})
export class TranslateConfigService {


  currentLang: any;

  constructor(
    private translate: TranslateService,
  ) {
    this.currentLang = localStorage.getItem('lang');
  }

  getDefaultLanguage(){
    if (this.currentLang) {
      this.translate.setDefaultLang(this.currentLang);
    } else {
      localStorage.setItem('lang',
this.translate.getBrowserLang()!);
      this.currentLang = this.translate.getBrowserLang();
      this.translate.setDefaultLang(this.currentLang);
    }
    return this.currentLang;
  }

  setLanguage(setLang: string) {
    this.translate.use(setLang);
    localStorage.setItem('lang', setLang);
  }

  getCurrentLang() {
    return localStorage.getItem('lang');
  }

}
```

¿Qué hemos hecho?

Inicialmente cargamos el lenguaje actual desde el almacenamiento local (local storage). La función coge el lenguaje por defecto y mira si existe en el almacenamiento local, si no existe entonces utiliza el lenguaje del dispositivo. La función **set** establece el lenguaje en el almacenamiento local. La función **get** se trae el lenguaje del almacenamiento local.

TABS

Vamos a configurar los tabs de la aplicación. Lo primero que vamos a hacer es definir la página con los tabs y establecer a tab1, tab2 y tab3 como hijos de tabs. En **tabs.page.html**:

```
<ion-tabs>

  <ion-tab-bar slot="bottom">
    <ion-tab-button tab="tab1">
      <ion-icon name="triangle"></ion-icon>
      <ion-label>{{"HOME" | translate}}</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="tab2">
      <ion-icon name="ellipse"></ion-icon>
      <ion-label>{{"CONTENT" | translate}}</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="tab3">
      <ion-icon name="square"></ion-icon>
      <ion-label>{{"ABOUT" | translate}}</ion-label>
    </ion-tab-button>
  </ion-tab-bar>

</ion-tabs>
```

Y en **tabs.routes.ts**:

```
import { Routes } from '@angular/router';
import { TabsPage } from './tabs.page';

export const routes: Routes = [
  {
    path: 'tabs',
    component: TabsPage,
    children: [
      {
        path: 'tab1',
        loadComponent: () =>
          import('../tab1/tab1.page').then((m) => m.Tab1Page),
      },
      {
        path: 'tab2',
        loadComponent: () =>
          import('../tab2/tab2.page').then((m) => m.Tab2Page),
      },
      {
        path: 'tab3',
        loadComponent: () =>
          import('../tab3/tab3.page').then((m) => m.Tab3Page),
      },
      {
        path: '',
```

```
        redirectTo: '/tabs/tab1',
        pathMatch: 'full',
      },
    ],
  },
  {
    path: '',
    redirectTo: '/tabs/tab1',
    pathMatch: 'full',
  },
];
```

El siguiente paso es añadir el TranslateModule a nuestro **tabs.page.ts** además del servicio:

```
import { Component, EnvironmentInjector, inject } from
'@angular/core';
import { IonicModule } from '@ionic/angular';
import {TranslateModule, TranslateService} from "@ngx-
translate/core";
import {TranslateConfigService} from "../services/translate-
config.service";

@Component({
  selector: 'app-tabs',
  templateUrl: 'tabs.page.html',
  styleUrls: ['tabs.page.scss'],
  standalone: true,
  imports: [IonicModule, TranslateModule],
})
export class TabsPage {
  public environmentInjector = inject(EnvironmentInjector);
  language: any;

  constructor(private translateConfigService:
TranslateConfigService, private translate: TranslateService) {
    this.translateConfigService.getDefaultLanguage();
    this.language =
this.translateConfigService.getCurrentLang();
  }
}
```

Tab1 Tab2 Tab3

Definimos las páginas que tendrán el contenido de nuestra App y su action sheet que controlará el idioma. En **tab1.page.ts, tab2.page.ts y tab3.page.ts**:

```
import { Component } from '@angular/core';
import {ActionSheetController, IonicModule} from '@ionic/angular';
import { ExploreContainerComponent } from '../explore-
container/explore-container.component';
import {TranslateConfigService} from "../services/translate-
config.service";
import {TranslateModule} from "@ngx-translate/core";

@Component({
  selector: 'app-tab1',
  templateUrl: 'tab1.page.html',
  styleUrls: ['tab1.page.scss'],
  standalone: true,
  imports: [IonicModule, ExploreContainerComponent,
TranslateModule],
})
export class Tab1Page {
  language: any;

  constructor(
    private translateConfigService: TranslateConfigService,
    public actionSheetController: ActionSheetController) {
    this.translateConfigService.getDefaultLanguage();
    this.language = this.translateConfigService.getCurrentLang();
  }

  async changeLanguage() {
    const actionSheet = await this.actionSheetController.create({
      header: 'Languages',
      buttons: [{
        text: 'English',
        icon: 'language-outline',
        handler: () => {
          this.language = 'en';
          this.translateConfigService.setLanguage('en');
        }
      }, {
        text: 'Spanish',
        icon: 'language-outline',
        handler: () => {
          this.language = 'es';
          this.translateConfigService.setLanguage('es');
        }
      }, {
        text: 'Arabic',
        icon: 'language-outline',
        handler: () => {
          this.language = 'ar';
          this.translateConfigService.setLanguage('ar');
        }
      }, {
        text: 'Cancel',
        icon: 'close',
        role: 'cancel',
        handler: () => {
          console.log('Cancel clicked');
```

```
        }
      }]
    });
    await actionSheet.present();

    const { role, data } = await actionSheet.onDidDismiss();
    console.log('onDidDismiss resolved with role and data', role,
data);
  }
}
```

Y su correspondiente html, **tab1.page.html**:

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      {{"HOME" | translate}}
    </ion-title>
    <ion-buttons slot="end">
      <ion-button (click)="changeLanguage()">
        <ion-icon name="language-outline"></ion-icon>
      </ion-button>
    </ion-buttons>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
  <ion-header collapse="condense">
    <ion-toolbar>
      <ion-title size="large">{{"HELLO" | translate}}</ion-
title>
    </ion-toolbar>
  </ion-header>

  <ion-card>
    <ion-card-content>
      {{"WELCOME" | translate}}
    </ion-card-content>
  </ion-card>
</ion-content>
```

tab2.page.html:

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      {{"CONTENT" | translate}}
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
  <ion-header collapse="condense">
    <ion-toolbar>
      <ion-title size="large">{{"ABOUT" | translate}}</ion-
```

```
title>
    </ion-toolbar>
  </ion-header>

  <ion-card>
    <ion-card-content>
      {{"SIMPLE_APP" | translate}}
    </ion-card-content>
  </ion-card>
</ion-content>
```

Y **tab3.page.html**:

```
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      {{"ABOUT" | translate}}
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true">
  <ion-header collapse="condense">
    <ion-toolbar>
      <ion-title size="large">{{"HELLO" | translate}}</ion-
title>
    </ion-toolbar>
  </ion-header>

  <ion-card>
    <ion-card-content>
      {{"MY_NAME" | translate}}
    </ion-card-content>
  </ion-card>
</ion-content>
```